

Module 1 — Behavior

How it works (plain language)

Module 1 looks at the patient's GPS history and figures out the handful of places they actually visit (home, the market, a relative's house). It does this in two steps:

1. **Clean the data** (`preprocess_gps`) — drop incomplete rows, smooth out GPS jitter with a Kalman filter, and smooth the speed readings.
2. **Find the places** (`cluster_places`) — run **DBSCAN** clustering with a ~50-metre radius. Any spot the patient stood near at least 5 times becomes a "known place"; one-off points are treated as noise and ignored.

For each place it records the centre point, how often it was visited, and the average time spent there.

Key code

```
# place_clustering.py – DBSCAN over GPS coordinates (haversine metric)
eps = 50 / EARTH_RADIUS_M          # ~50 metre radius, expressed in radians
db = DBSCAN(eps=eps, min_samples=5, metric="haversine").fit(np.radians(coords))
df["cluster"] = db.labels_

for cluster_id in set(db.labels_):
    if cluster_id == -1:              # -1 = noise, skip
        continue
    cluster_data = df[df["cluster"] == cluster_id]
    results.append({
        "cluster_id": int(cluster_id),
        "latitude": cluster_data["latitude"].mean(),
        "longitude": cluster_data["longitude"].mean(),
        "visit_frequency": len(cluster_data),
        "avg_stay_time": round(stay_minutes.get(int(cluster_id), 0.0), 1),
    })
```

Real input → output

We fed in 19 raw GPS rows: 8 fixes clustered around a "home" location, 8 around a "market" location, and 3 scattered one-off points.

```
INPUT (raw GPS history, 19 rows – first 6 shown):
latitude longitude speed timestamp
13.746018 100.534059 0.245172 2026-06-01 08:00:00
13.745960 100.533970 0.101671 2026-06-01 08:02:00
13.745929 100.534048 0.201559 2026-06-01 08:04:00
13.745995 100.533971 0.371377 2026-06-01 08:06:00
13.745965 100.533992 0.294150 2026-06-01 08:08:00
13.746008 100.534074 0.208477 2026-06-01 08:10:00

OUTPUT (clustered significant places):
[
  {
    "cluster_id": 0,
```

```

    "latitude": 13.745984131926143,
    "longitude": 100.53402560952179,
    "visit_frequency": 8,
    "avg_stay_time": 14.0
  },
  {
    "cluster_id": 1,
    "latitude": 13.75098143461485,
    "longitude": 100.53999137411142,
    "visit_frequency": 8,
    "avg_stay_time": 14.0
  }
]

```

Result: the two real places were found (8 visits each), and the 3 scattered points were correctly discarded as noise.

Module 2 — Prediction

Module 2 takes the patient's *recent* movement plus the known places from Module 1 and answers four questions.

2.1 Which known place am I at? (cluster matcher)

How it works: measures the great-circle (haversine) distance from the current point to every known place; returns the nearest one if it's within 150 m, plus a "familiarity" score (how often that place is visited, relative to the most-visited place).

```

def find_nearest_cluster(lat, lng, known_places, max_distance_km=0.15):
    best_id, best_dist = None, float('inf')
    for place in known_places:
        dist = haversine_km(lat, lng, place['latitude'], place['longitude'])
        if dist < best_dist:
            best_dist, best_id = dist, place['cluster_id']
    return best_id if best_dist <= max_distance_km else None

```

```

INPUT  known_places + query point (on place 0)
OUTPUT:
{
  "nearest_cluster_id": 0,
  "familiarity_cluster0": 1.0,
  "familiarity_cluster1": 0.3,
  "haversine_km_place0_to_place1": 0.8539
}

```

2.2 Is the patient wandering? (wandering detection)

How it works: an **Isolation Forest** is trained on the patient's normal walking (straight-ish, steady speed). New movement is scored 0–1 on how "anomalous" it looks — lots of direction changes and circling in a small area push the score up. Above a mild threshold it flags wandering. If the model isn't

trained it falls back to simple rules.

```
INPUT  fit on 120 normal-walk points; then detect two 20-point windows:
{ "status": "fitted", "n_windows": 12, "n_gps_records": 120 }

OUTPUT normal walk:
{ "wandering_score": 0.48, "wandering_level": "normal", "wandering_detected": false }

OUTPUT wandering walk:
{ "wandering_score": 0.553, "wandering_level": "mild", "wandering_detected": true }
```

Result: the straight walk stayed "normal"; the circling walk was detected as mild wandering.

2.3 Stopped normally, or confused? (stop/confusion classification)

How it works: when the patient stops, a **Gradient Boosting** classifier decides between a *normal* stop (short, near a known place, on a familiar route) and a *confused* stop (long, far from anywhere familiar, off-route, lots of turning before stopping).

```
INPUT  train: { "status": "trained", "n_samples": 100 }

OUTPUT normal stop (short, AT a known place):
{ "status": "normal", "confidence_score": 0.003 }

OUTPUT confused stop (15 min, far, off-route):
{ "status": "confused", "confidence_score": 0.997 }
```

Result: a short stop at a known place reads as normal; a long stop far from anywhere familiar reads as confused with 99.7% confidence.

2.4 Which way will they go? (route prediction)

How it works: a **Hidden Markov Model** learns how the patient moves between places (transition probabilities). **Viterbi decoding** maps recent GPS onto the most likely place sequence, then the route is extended to the predicted destination. The predicted route is compared to past routes with **DTW** to score how "familiar" it is.

```
INPUT  fit on 96 history points (home->market daily, 8 days):
{ "status": "fitted", "n_clusters": 2, "n_transitions": 15, "n_historical_routes": 8 }

OUTPUT predict route from home -> market (cluster 1):
{
  "status": "ok",
  "predicted_route": [
    { "latitude": 13.746, "longitude": 100.534, "cluster_id": 0 },
    { "latitude": 13.751, "longitude": 100.54, "cluster_id": 1 }
  ],
  "similarity_score": 1.0,
  "route_familiar": true
}
```

Result: the model correctly predicts the home→market route and recognises it as familiar (similarity 1.0).

Note: Module 2 also has a `DestinationPredictor` (an LSTM neural net). It is skipped in this document because it requires TensorFlow, which is not installed in this environment.

Module 3 — Risk

Module 3 is the heart of the system: it turns all the Module 1 & 2 signals into a single **0–100 risk score**, decides whether to raise an emergency, and handles the case where GPS goes silent.

3.1 Normalize the signals (0–1)

Every raw signal is mapped onto a 0–1 scale so they can be combined fairly. Distance off-route is divided by a 500 m ceiling; "familiarity" is inverted into "unfamiliarity" (a familiar place is *low* risk).

```
INPUT:  { "route_deviation_m": 320.0, "wandering": 0.7, "danger_zone": true, "familiarity": 0.3 }
OUTPUT: { "route_deviation": 0.64, "wandering": 0.7, "danger_zone": 1.0, "unfamiliarity": 0.7 }
```

3.2 Compute the risk score

How it works: a fixed weighted sum (weights total 1.0), scaled to 0–100. The output includes a Low/Medium/High level and a per-factor breakdown that always sums back to the headline score.

```
WEIGHTS = {"route_deviation": 0.30, "wandering": 0.25, "confusion": 0.20,
           "danger_zone": 0.15, "unfamiliarity": 0.10}

contributions = {k: round(w * factors[k] * 100, 1) for k, w in WEIGHTS.items()}
risk_score = round(sum(contributions.values()), 1)
# level: <50 low, 50-79 medium, >=80 high
```

```
INPUT  normalized factors:
{ "route_deviation": 0.64, "wandering": 0.70, "confusion": 0.40,
  "danger_zone": 0.0, "unfamiliarity": 0.70 }

OUTPUT:
{
  "risk_score": 51.7,
  "risk_level": "medium",
  "contributions": {
    "route_deviation": 19.2, "wandering": 17.5, "confusion": 8.0,
    "danger_zone": 0.0, "unfamiliarity": 7.0
  }
}
```

3.3 Emergency decision

Rule: raise an emergency if the patient is in a **danger zone** or the risk score is **above 80**. A danger zone always takes precedence.

```
INPUT risk_score=85.0, danger_zone=False:
{ "emergency": true, "reason": "high_score", "severity": "high", "alert_type": "emergency" }

INPUT risk_score=50.0, danger_zone=True:
{ "emergency": true, "reason": "danger_zone", "severity": "critical", "alert_type": "geofence" }

INPUT risk_score=40.0, danger_zone=False:
{ "emergency": false, "reason": null, "severity": "normal", "alert_type": "none" }
```

3.4 GPS failure handling

How it works: if the gap since the last GPS fix exceeds 10 minutes (600 s), GPS is treated as lost and the last known position is packaged up for the (future) search module. The design is *safety-biased* — anything it can't evaluate is treated as lost.

```
INPUT last fix 300 s ago (threshold 600 s):
{ "gps_lost": false, "gap_seconds": 300.0,
  "last_known": { "latitude": 13.7563, "longitude": 100.5018, "recorded_at": "2026-06-26T11:55:00" } }

INPUT last fix 845 s ago:
{ "gps_lost": true, "gap_seconds": 845.0,
  "last_known": { "latitude": 13.7563, "longitude": 100.5018, "recorded_at": "2026-06-26T11:45:55" } }
```

3.5 The orchestrator (collect risk factors)

How it works: `collect_risk_factors` is the glue — it fits the Module 2 detectors on the patient's history and assembles the five raw factors for scoring. When a signal can't be computed it falls back to safe, cautious defaults.

```
INPUT 10-day history + 6 recent fixes near MARKET (current = market):
{ "history_points": 160, "recent_points": 6, "current": [13.751, 100.54] }

OUTPUT raw factors:
{
  "route_deviation": 0.0,
  "wandering": 0.0,
  "confusion": 0.0,
  "danger_zone": false,
  "familiarity": 0.3,
  "_meta": {
    "wandering_status": "ok", "route_status": "ok", "destination_cluster_id": 0,
    "stopped": false, "avg_recent_speed_ms": 1.2, "matched_cluster_id": 1,
    "defaults_fired": []
  }
}
```

Result: the patient is on a known route at a known place, so every factor is low — exactly what we'd expect for someone safely at the market.

Module 3 — Live database test (GET /api/risk)

This is the **full end-to-end path** through the real API endpoint and the real database. To prove it works without leaving any test data behind, the whole thing ran inside **one database transaction that was rolled back** at the end.

What it does, step by step: fetch the patient's profile + GPS history → run the Module 2 detectors + Module 3 scoring → save a risk score → save an alert if an emergency fires → return the response.

```
# Inside ONE real DB transaction (flush only – never committed):
user = await crud.create_user(session, firebase_uid=DEMO_UID, name="Demo Patient", role="patient")
await crud.upsert_behavioral_profile(session, user.id, known_places=json.dumps(places))
# ... 16 GPS fixes seeded ...
r1 = await get_risk(patient_id=user.id, lat=None, lng=None, db=session) # normal
r2 = await get_risk(patient_id=user.id, lat=zone_lat, lng=zone_lng, db=session) # danger zone
await session.rollback() # <-- discard EVERYTHING: seed + risk scores + alerts
```

Input

```
INPUT seeded patient (id=10) with profile + 16 GPS fixes (NOT committed):
{ "patient_id": 10, "known_places": 2, "gps_points": 16 }
```

Output 1 — normal call (patient at the market)

```
OUTPUT GET /api/risk/10 (near MARKET, no override):
{
  "patient_id": 10,
  "status": "ok",
  "message": "Risk 24.5% (low).",
  "risk_score": 24.5,
  "risk_level": "low",
  "contributions": {
    "route_deviation": 0.0, "wandering": 17.5, "confusion": 0.0,
    "danger_zone": 0.0, "unfamiliarity": 7.0
  },
  "wandering_detected": true,
  "gps_available": true,
  "emergency": false,
  "reason": null
}
```

Output 2 — location overridden into a danger zone

```
OUTPUT GET /api/risk/10?lat=13.7700&lng=100.5550 (danger zone: Major highway interchange (demo)):
{
  "patient_id": 10,
  "status": "ok",
  "message": "Risk 72.5% (medium).",
  "risk_score": 72.5,
  "risk_level": "medium",
  "contributions": {
    "route_deviation": 30.0, "wandering": 17.5, "confusion": 0.0,
    "danger_zone": 15.0, "unfamiliarity": 10.0
  },
  "wandering_detected": true,
  "gps_available": true,
  "emergency": true,
  "reason": null
}
```

```
"reason": "danger_zone"
}
```

A geofence / critical alert was written for this case (and then rolled back).

Proof that nothing persisted

```
>>> session.rollback() called – all seeded + computed rows discarded.
```

```
VERIFY after rollback – demo patient lookup:
{ "demo_user_id": null, "persisted": false }
```

Result: the live endpoint computed, scored, decided, and (would have) alerted correctly — and after rollback the demo patient does not exist in the database. The database is exactly as it was before the test.

Module 4 — Search Area

How it works (plain language)

When GPS goes silent, Module 4 estimates *where to look*. It is wired together by the API router (`api/search_area.py`) — the four AI files don't import each other. The pipeline runs in four steps:

1. **Anchor + base radius** (`last_known_position.py`) — take the last known fix (preferring the Kalman-smoothed coordinates) and turn "how long they've been missing" into a base radius: `radius = speed × time` (Distance = Speed × Time). If speed is unknown it falls back to a cautious 1.4 m/s walking pace.
2. **Adjust the radius** (`search_radius_adjustment.py`) — widen or tighten the circle using *behavioural* signals, **not** any dementia-stage label: ×1.5 if no familiar place sits inside the radius, ×1.3 if the wandering score is high (≥ 0.75), ×0.8 if it's low (≤ 0.30). Multipliers compound.
3. **Simulate the paths** (`movement_path_simulation.py`) — a **Monte Carlo** simulation scatters thousands of possible endpoints inside the circle (70% biased toward the directions the patient historically travels, 30% uniform), plus a simplified **A*** that draws great-circle routes toward the most-visited familiar places within range.
4. **Estimate the zones** (`probability_area_estimation.py`) — run **Kernel Density Estimation** (SciPy `gaussian_kde`) over those endpoints on a 50×50 grid and split the area into **High / Medium / Low** probability zones by percentile (High $\geq p70$, Medium $p40$ – $p70$, Low $< p40$). If SciPy is unavailable it falls back to distance-based thirds.

It reuses Module 3's `detect_gps_gap()` to confirm the patient is actually missing, and writes a `gps_loss` alert (severity `high`) when they are.

Response shape

`GET /api/search-area/{patient_id}` returns a `SearchAreaResponse`. The field names below are the real contract (`app/models/search_area.py`); the numbers are illustrative, not a captured run.

```
{
  "patient_id": 10,
  "status": "ok", # "ok" | "no_data" | "gps_active"
  "message": "Search area estimated (radius 315 m).",
  "last_known_location": { "latitude": 13.7563, "longitude": 100.5018, "recorded_at": "2026-06-26T11:00:00"},
  "search_radius_meters": 210.0, # base radius (Distance = Speed × Time)
  "adjusted_radius_meters": 315.0, # after ×1.5 / ×1.3 / ×0.8 adjustments
  "adjustment_reason": "no familiar place within base radius (×1.5)",
  "high_probability_zone": [ { "latitude": 13.7570, "longitude": 100.5025, "probability": 0.92 } ],
  "medium_probability_zone": [ { "latitude": 13.7555, "longitude": 100.5010, "probability": 0.55 } ],
  "low_probability_zone": [ { "latitude": 13.7540, "longitude": 100.4998, "probability": 0.18 } ],
  "target_locations": [
    { "name": "market", "latitude": 13.7510, "longitude": 100.5400, "visit_frequency": 8, "distance_m": 120.0 },
  ],
  "familiar_paths": [
    { "place_name": "market", "visit_frequency": 8, "distance_m": 120.0,
      "waypoints": [ [13.7563, 100.5018], [13.7536, 100.5209] ] },
  ],
  "grid_bounds": { "lat_min": 13.753, "lat_max": 13.760, "lng_min": 100.499, "lng_max": 100.505 }
}
```

Result: from a last known position and a time-missing value, Module 4 produces ranked High/Medium/Low search zones plus the familiar places worth checking first — the hand-off that Module 3's `gps_failure_handling` was designed to feed. When GPS is still live, the endpoint short-circuits with `status: "gps_active"` and no zones.

Module 5 — Recommend

How it works (plain language)

If a caregiver needs to know *where the patient probably is*, Module 5 scores every known place by blending three signals — how often it's visited (**frequency**), how close it is right now (**proximity**), and how long they usually stay (**familiarity**) — then returns the top suggestions, highest confidence first.

```
# recommendation_generation.py – transparent, rule-based blend
WEIGHTS = {"frequency": 0.45, "proximity": 0.35, "familiarity": 0.20, "time_match": 0.0}
proximity = 1.0 / (1.0 + distance_km) # closer => higher
confidence = sum(WEIGHTS[f] * factors[f] for f in active) / sum(WEIGHTS[f] for f in active)
```

Real input → output

```
INPUT profile (3 known places) + current location near place 1
```

```
OUTPUT ranked recommendations (top 3):
```

```
[
  { "cluster_id": 0, "confidence": 0.847,
    "factors": { "frequency": 1.0, "proximity": 0.5628, "familiarity": 1.0, "time_match": 0.0 },
    "location_used": true },
  { "cluster_id": 1, "confidence": 0.5098,
    "factors": { "frequency": 0.3, "proximity": 0.9281, "familiarity": 0.25, "time_match": 0.0 },
    "location_used": true },
```

```
{ "cluster_id": 2, "confidence": 0.2185,  
  "factors": { "frequency": 0.125, "proximity": 0.3922, "familiarity": 0.125, "time_match": 0.0 },  
  "location_used": true }  
]
```

Result: cluster 0 wins — even though cluster 1 is physically closer, cluster 0's high visit frequency and long typical stay make it the most likely location.

Note — learned ranker: the blend above is the transparent *rules* path and the default when no model is trained. Module 5 also ships a **learned pointwise ranker** (`ranker.py`, a scikit-learn `HistGradientBoostingClassifier`). When a per-patient model artifact (`ranker_patient_{id}.pkl`) exists, the API loads it and uses its $P(\text{chosen})$ as the confidence instead — results are flagged `scorer="ml"`, falling back to `scorer="rules"` otherwise (never faked as ML). The model is trained and validated offline in `evaluation.py` (temporal split, paired bootstrap CIs, and it must beat both the majority-class and context-blind baselines before it's accepted).

Summary

Module	Demonstrated	Result
1 — Behavior	preprocess + DBSCAN clustering	2 places found from noisy GPS ✓
2 — Prediction	cluster match, wandering, confusion, route	all four behave correctly ✓
3 — Risk	normalize, score, emergency, GPS gap, orchestrator	correct scores & decisions ✓
3 — Risk (live DB)	real GET <code>/api/risk</code> , rolled back	full path works, nothing persisted ✓
4 — Search Area	last known → radius → Monte Carlo → KDE zones	High/Medium/Low zones + target places ✓
5 — Recommend	context → score (rules or learned ranker) → top-N	sensible ranking ✓

How to reproduce

The repository also ships an automated test suite (87 tests) covering all of the above, runnable with no PostgreSQL, Firebase, or TensorFlow required:

```
cd backend  
pip install -r requirements-dev.txt  
python -m pytest
```

Note: the `DestinationPredictor LSTM` (Module 2) requires TensorFlow, which is not installed here, so it is excluded from both this document and the automated suite.